

Plataforma de Gestión Escolar

Colegio Salesiano San José (CSSJ)

Documentación Técnica, Diccionario de Datos y Guía de Traspaso

Proyecto: gestion-escolar · Kyuuri Studios

Versión del documento: 1.0 · Fecha: 10 de julio de 2026

Autor del sistema: Carlos Castañeda (CarlosCastaneda04)

Índice

Índice.....	2
1. Resumen ejecutivo.....	4
2. Stack tecnológico y arquitectura.....	4
2.1 Tecnologías	4
2.2 Estructura del repositorio	4
2.3 Servicios (src/services/)	5
3. Modelo de usuarios y autenticación.....	5
3.1 Roles y "candado" del panel admin	6
4. Módulos implementados (lo que YA existe)	6
4.1 Sitio web público.....	6
4.2 Módulo de Admisión (nuevo ingreso) — flujo completo.....	6
4.3 Módulo de Reingreso (antiguo ingreso)	7
4.4 Panel administrativo	8
4.5 Cloud Functions (backend).....	8
5. Identificadores, códigos y formatos clave.....	9
5.1 Catálogos de códigos de grado (¡hay DOS, no confundirlos!).....	9
6. Mapa de rutas de la aplicación	9
6.1 Rutas públicas	9
6.2 Rutas administrativas.....	10
7. Diccionario de datos (Firestore).....	10
7.1 Resumen de colecciones	10
7.2 admissions (solicitud de admisión)	11
7.3 accessCodes (códigos de acceso).....	12
7.4 admissionDocuments (documentos del expediente)	13
7.5 admissionEvaluations (evaluaciones individuales)	13
7.6 evaluationTemplates (plantillas de examen)	14
7.7 alumnos (expediente maestro — AlumnoReingreso).....	14
7.8 reingresoPayments (pagos y contratos de reingreso)	15
7.9 configuracion/matricula (ciclo escolar y aranceles).....	15
7.10 Otras colecciones	16
7.11 Cloud Storage (estructura de archivos).....	16

8. Modelo de seguridad actual	16
8.1 Reglas de Firestore (firestore.rules).....	16
8.2 Requisitos operativos para que el modelo funcione	17
8.3 Debilidades conocidas y aceptadas (por diseño de la Fase actual).....	17
9. Lo que FALTA (backlog para quien continúe).....	17
9.1 Prioridad ALTA — Seguridad (Fase 2c).....	17
9.2 Prioridad MEDIA — Datos y operación	17
9.3 Prioridad MEDIA/LARGO PLAZO — Módulos no construidos	18
9.4 Calidad de ingeniería.....	18
10. Guía de traspaso (para el siguiente desarrollador).....	18
10.1 Accesos que necesitas pedir	18
10.2 Puesta en marcha local	19
10.3 Despliegue.....	19
10.4 Operación anual: abrir un nuevo ciclo escolar.....	19
10.5 Mapa "quiero cambiar X → toco Y"	19
10.6 Convenciones y "gotchas" del proyecto	20
11. Anexo — Historial y estado a la fecha.....	20

1. Resumen ejecutivo

Esta plataforma es el sitio web institucional y el sistema de matrícula en línea del Colegio Salesiano San José (El Salvador). Cubre de punta a punta dos procesos de negocio reales y en producción:

- **ADMISIÓN** (nuevo ingreso): un aspirante recibe un código de acceso, llena su solicitud y expediente digital, sube documentos, realiza evaluaciones (psicológica, académica en línea, inglés), recibe un dictamen, y si es aprobado se le generan credenciales institucionales (carnet, correo Microsoft 365 real con licencia A1 y Teams), paga la matrícula con talonario NPE y firma contrato. Al aprobar su contrato pasa a ser alumno del colegio.
- **REINGRESO** (antiguo ingreso): los 2,196 alumnos existentes (migrados desde SQL Server el 07-jul-2026) entran con carnet + PIN, ratifican/actualizan sus datos, imprimen su talonario de pago NPE, suben su comprobante de pago y su contrato firmado, y colecturía los revisa y aprueba.

Además incluye el sitio público del colegio (noticias, historia, autoridades, calendario, etc.) editable desde un gestor administrativo, y un panel de administración protegido por roles (admin / editor_web) con re-autenticación ("candado") para las áreas sensibles.

Lo que NO existe todavía (ver sección 9): gestión académica del día a día (notas, asistencia, cursos, docentes, portal de padres), cobro de mensualidades, notificaciones por correo, y el endurecimiento final de seguridad del flujo de aspirantes.

2. Stack tecnológico y arquitectura

2.1 Tecnologías

Capa	Tecnología	Notas
Frontend	React 19 + TypeScript + Vite 8	SPA con react-router-dom 7. Estilos: Tailwind CSS 4 + CSS propio por página. Animaciones AOS.
Componentes	react-barcode, react-calendar	Código de barras del talonario NPE y calendario académico.
Backend (BaaS)	Firebase (plan Blaze)	Hosting, Firestore, Storage, Authentication, Cloud Functions v2 (Node, región us-central1).
Integración externa	Microsoft Graph API	Creación real de cuentas Microsoft 365 de estudiantes (Azure AD App Registration, auth app-only).
Base de datos	Cloud Firestore (NoSQL)	Ver diccionario de datos (sección 7).
Archivos	Cloud Storage	Documentos de admisión, comprobantes de pago, contratos firmados, portadas del sitio.

2.2 Estructura del repositorio

```
gestion-escolar/
├─ src/
│   └─ App.tsx           → todas las rutas (mapa en sección 6)
│   └─ auth/             → adminAuth.ts (roles admin), studentSession.ts
│   (sesión alumno)
│   └─ components/       → home/ (secciones del sitio), layout/ (Navbar,
Footer,
│   └─ ProtectedRoute, CandadoRecursos), admin/,
```

admisiones/	
pages/	
home/	→ páginas públicas informativas
admisiones/	→ acceso con código, portal del aspirante, examen en línea
reingreso/	→ login, formulario de ratificación, pasos, talonario
admin/	→ todo el panel administrativo
shared/	→ ContratoImpresion (contrato imprimible)
services/	→ acceso a Firestore/Storage/Functions (1 servicio por dominio)
types/	→ index.ts (admisión/matricula), reingreso.ts (alumnos antiguos)
hooks/	→ useAnioMatricula, usePortada, useUserRole
utils/	→ npeGenerator, credentialsGenerator, alumnoSqlParser, formatters, imageCompression, imageValidation
data/	→ elSalvadorGeo.ts, alumnosParaImportar.ts (filas SQL corregidas)
firebase/config.ts	→ inicialización (lee variables VITE_ del archivo .env)
functions/	→ Cloud Functions (provisionM365User, loginReingreso)
firestore.rules	→ reglas de seguridad de Firestore
storage.rules	→ reglas de seguridad de Storage
firebase.json	→ hosting (dist/), functions, rules

2.3 Servicios (src/services/)

Servicio	Responsabilidad
admissionService.ts	CRUD de solicitudes de admisión, generación de carnet (generateStudentCarnet), aprobación con credenciales (approveForEnrollment), rechazo.
accessCodeService.ts	Emitir, validar y consumir códigos de acceso de admisión (con año de matrícula).
admissionDocumentService.ts	Subida y revisión de documentos del expediente (con versiones y re-subida).
admissionFinanceService.ts	Comprobante de pago y contrato firmado del aspirante (dentro del doc de admisión).
evaluationService.ts	Fases, evaluadores, jornadas, fechas globales, evaluaciones individuales, plantillas de examen y calificación.
enrollmentService.ts	Proceso de matrícula formal (enrollments/...). PARCIALMENTE CÓDIGO MUERTO: createEnrollment y GRADE_FEES no se usan.
alumnoService.ts	Alumnos de antiguo ingreso: login, lectura/actualización, ratificación, importación por lotes (400), crearAlumnoDesdeAdmision.
reingresoFinanceService.ts	Pagos y contratos firmados de reingreso (colección reingresoPayments, id = carnet).
configService.ts	Ciclo escolar activo, fecha límite y aranceles por grado (configuracion/matricula), con caché de sesión.
microsoftProvisioningService.ts	Cliente del callable provisionM365User (crea la cuenta M365 real).
personRegistryService.ts	STUB / simulado: devuelve IDs falsos. Pensado para un futuro registro institucional de personas.

3. Modelo de usuarios y autenticación

La plataforma tiene TRES tipos de usuario con mecanismos de acceso distintos. Entender esto es clave para leer las reglas de Firestore.

Usuario	Cómo entra	Sesión	Qué puede hacer
Administrador	Correo + contraseña (Firebase Auth, proveedor "password") en /admin/login	Firebase Auth	Panel completo según rol. Rol en colección usuarios_admin (doc id = correo en minúsculas): "admin" (todo) o "editor_web" (solo gestor de página web). Sin doc → editor_web por defecto (mínimo privilegio).
Aspirante (nuevo ingreso)	Código de acceso emitido por el colegio en /solicitud-admision	SIN sesión (anónimo)	Crear y dar seguimiento a SU solicitud en /mi-solicitud, subir documentos, rendir examen en línea, subir pago y contrato.
Alumno (reingreso)	Carnet + PIN en /reingreso/login. El PIN se valida EN EL SERVIDOR (Cloud Function loginReingreso) y se emite un custom token con uid = carnet y claim role=student	Firebase Auth (proveedor "custom")	Leer/actualizar únicamente SU documento en la colección alumnos; ratificar datos, talonario, pago y contrato.

3.1 Roles y "candado" del panel admin

- Lógica central en src/auth/adminAuth.ts. ProtectedRoute acepta props rol y candado.
- Candado = re-autenticación (vuelve a pedir la contraseña) antes de entrar a Recursos Internos; válido 30 minutos por pestaña (sessionStorage). UI en src/components/layout/CandadoRecursos.tsx.
- Bootstrap: el array ADMININS_BOOTSTRAP en adminAuth.ts permite marcar correos como admin sin doc en usuarios_admin.
- La contraseña temporal M365 de los estudiantes NO vive en la admisión: está en studentCredentials/{admissionId} (solo admin) y la pantalla la recalcula de forma determinista con buildTempPassword(carnet).

4. Módulos implementados (lo que YA existe)

4.1 Sitio web público

Páginas informativas del colegio, con contenido editable desde /admin/institucional (colecciones institucional, noticias, eventos, autoridades; portadas en Storage /portadas). Páginas: Inicio (hero con video institucional, niveles educativos, noticias, valores, estadísticas, contacto), Nosotros, Historia, Ubicación, Autoridades, Mundo Salesiano, Modelo Educativo, Directorio Institucional, Noticias, Calendario Académico, Proceso de Inscripción y Política de Privacidad.

4.2 Módulo de Admisión (nuevo ingreso) — flujo completo

1. Emisión del código: el admin genera un código en /admin/codigos, asignado a un aspirante/encargado concretos y a un AÑO de matrícula (ordinaria = ciclo configurado; extraordinaria = año en curso). El año viaja: código → admisión (enrollmentYear) → carnet → contrato.

2. Solicitud: la familia canjea el código en /solicitud-admision, acepta la política de privacidad (privacyConsent con versión) y llena la solicitud (datos del aspirante, encargado, dirección con geografía de El Salvador).
3. Expediente digital: en /mi-solicitud completa el expediente (NIE, salud, sacramentos, familia, transporte, sostenedor económico) — los mismos datos que tiene un alumno antiguo.
4. Documentos: sube partida de nacimiento, notas, foto, carnet de vacunas, identificación, solvencia, etc. El admin los revisa (aprueba/rechaza con motivo; la familia puede re-subir, se guardan versiones).
5. Evaluaciones: el admin habilita exámenes (valida que los documentos REQUERIDOS del grado estén aprobados). Tipos: psicológica, académica (examen en línea con plantillas por grado construidas en /admin/evaluaciones/constructor), inglés (solo 7° en adelante, programable), entrevista psicológica. Hay jornadas y fechas globales por año.
6. Examen en línea: el aspirante lo rinde en /portal/examen/:evaluationId (preguntas de opción múltiple, con imágenes de apoyo, por materia). Se registran respuestas, asistencia, reprogramaciones.
7. Dictamen: resolución de entrevista (fortalezas, áreas de mejora, recomendaciones) y decisión final (aprobado/rechazado, con motivos y mensaje; el rechazo puede permitir ver resultados).
8. Aprobación y credenciales (Fase 1): en /admin/aprobacion se genera carnet (AÑO+GRADO+ORDEN), correo institucional (nombre.apellidoAÑO@salesianosanjose.edu.sv, con des-duplicado) y contraseña temporal (Cssj<carnet>!).
9. Provisión M365 (Fase 2, FUNCIONANDO): botón "Provisionar en M365" llama a la Cloud Function provisionM365User, que crea la cuenta real vía Microsoft Graph, asigna licencia Office 365 A1 for students (Teams) y marca provisioningStatus. Es idempotente y tolera fallo de licencia sin revertir la cuenta.
10. Matrícula: la familia imprime el talonario NPE (con código de barras y dígito verificador), paga, sube el comprobante (colecturía lo revisa en /admin/colecturia) y sube el contrato firmado (se revisa en /admin/contratos-firmados).
11. Alta como alumno: al aprobar el contrato firmado, alumnoService.crearAlumnoDesdeAdmision copia TODO el expediente a la colección alumnos (PIN = NIE o últimos 4 del carnet) y la admisión queda en status "enrolled". El siguiente ciclo, ese alumno entra por el portal de reingreso.

4.3 Módulo de Reingreso (antiguo ingreso)

- Datos base: 2,196 alumnos migrados desde la tabla dbo.alumno de SQL Server (cssj_db) el 07-jul-2026. Importador en /admin/importar-alumnos: se sube el volcado TSV (datos.sql, 60 columnas), se parsea en el navegador (src/utls/alumnoSqlParser.ts) y se escribe por lotes de 400. Filas rotas se corrigen a mano en src/data/alumnosParaImportar.ts (prioridad por carnet).
- Grados calculados: el grado actual y el grado a matricular se calculan avanzando desde grado_ingreso los años transcurridos hasta el ciclo activo (asume promoción continua). Los que terminaron quedan "Graduado" (998 de 2,196; 1,198 matriculables). El campo opcional grado_actual por fila permite corregir repitencias. El documento guarda anioCalculo y al leer se avanza automáticamente si el ciclo activo es mayor: cambiar de año NO requiere re-importar.
- Flujo del alumno: login (carnet + PIN) → formulario de ratificación de datos (incluye datos de facturación DUI/NIT del sostenedor, que no venían de SQL) → pasos de matrícula: talonario NPE

→ subir comprobante de pago → descargar contrato → subir contrato firmado. Pago y contrato son reales (colección reingresoPayments).

- Revisión administrativa: /admin/colecturia (comprobantes por revisar, pagos revisados, precios y cuotas por grado, ciclo escolar) y /admin/contratos-firmados (unifica contratos de nuevo ingreso y reingreso).

4.4 Panel administrativo

Pantalla	Ruta	Qué hace
Dashboard	/admin/dashboard	Portal central de selección de módulos.
Gestor de Página Web	/admin/institucional	Edita contenido público: misión/visión, contactos, portadas, noticias, eventos, autoridades. Accesible al rol editor_web.
Recursos Internos	/admin/recursos	Menú de los módulos académicos sensibles (todos con rol admin + candado).
Solicitudes	/admin/solicitudes	Bandeja de admisiones: expedientes, documentos, habilitar exámenes, seguimiento.
Evaluaciones	/admin/evaluaciones	Programar/calificar evaluaciones, jornadas, fechas globales, habilitar examen en línea, asistencia.
Constructor de exámenes	/admin/evaluaciones/constructor	Crear plantillas de examen académico por grado (preguntas, opciones, imágenes, materias).
Aprobación de matrícula	/admin/aprobacion	Decisión final, generación de credenciales, provisión M365, rechazo con motivo.
Colecturía	/admin/colecturia	Revisión de comprobantes (nuevo ingreso + reingreso), historial, aranceles por grado y configuración del ciclo escolar.
Códigos de acceso	/admin/codigos	Emitir códigos de admisión con año de matrícula, asignación nominal, usos y expiración.
Contratos (plantilla)	/admin/contratos	Configura la plantilla/datos del contrato (documento configuracion).
Contratos firmados	/admin/contratos-firmados	Revisión y aprobación de contratos firmados; al aprobar, el aspirante se convierte en alumno.
Importar alumnos	/admin/importar-alumnos	Migración SQL → Firestore de alumnos antiguos.

4.5 Cloud Functions (backend)

Función	Tipo	Qué hace
provisionM365User	onCall (us-central1)	Crea la cuenta Microsoft 365 real del estudiante vía Graph (app-only), asigna licencia A1 (AZURE_LICENSE_SKU) y actualiza la admisión. Exige llamador autenticado con correo del dominio institucional (ADMIN_EMAIL_DOMAIN). Idempotente.
loginReingreso	onCall (us-central1)	Valida carnet + PIN en el servidor (el PIN nunca viaja al navegador), verifica estado ACTIVO y emite custom token (uid = carnet, claim role=student). Mensaje de error genérico para no revelar si el carnet existe.

Secretos (firebase functions:secrets:set): AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET. Parámetros: AZURE_LICENSE_SKU = 314c4481-f395-4525-be8b-2ec4bb1e9d91 (Office 365 A1 for students) en functions/.env.gestion-escolar-cssj; ADMIN_EMAIL_DOMAIN (default salesianosanjose.edu.sv). Requisito Azure: App Registration con permiso de aplicación User.ReadWrite.All con admin consent.

5. Identificadores, códigos y formatos clave

Identificador	Formato	Ejemplo / notas
Carnet	AÑO(4) + GRADO(2) + ORDEN(2) = 8 dígitos	20260305 = ingresó 2026, Preparatoria (16... ver catálogo), correlativo 05. generateStudentCarnet consulta admissions Y alumnos para no duplicar (usa max+1).
PIN de reingreso	NIE del alumno; si no tiene, últimos 4 dígitos del carnet	Personalizable con el campo pin. Se valida solo en el servidor.
Correo institucional	nombre.apellido<año>@salesianosanjose.edu.sv	carlos.castaneda2026@... Si está tomado: nombre.apellido2026.2@... (buildUniqueEmail).
Contraseña temporal M365	Cssj<carnet>!	Cssj20260305! Determinista (recalculable); forceChangePasswordNextSignIn=true al provisionar.
NPE (talonario)	Estructura con monto (4 enteros + 2 decimales) y dígito verificador	Generado en src/utills/npeGenerator.ts; usa el carnet de la admisión cuando existe y el arancel del grado (getArancelParaGrado).
Código de acceso	Documento en accessCodes	Con año, grado, usos máximos, expiración y asignación nominal (aspirante + encargado).

5.1 Catálogos de códigos de grado (¡hay DOS, no confundirlos!)

Existen dos catálogos distintos de códigos de grado. Confundirlos es el error más fácil de cometer en este sistema:

Grado	Código en CARNET (tabla grado de SQL / CARNET_GRADE_CODES)	Código NPE (npeGenerator)
Kinder 4 / Kinder 5 / Preparatoria	14 / 15 / 16	01 / 02 / 03
1° a 9° Grado	01 ... 09	11 ... 19
1° / 2° Bachillerato general	10 / 11	21 / 22
Técnico Diseño Gráfico (1°-3°)	31 / 32 / 33	31 / 32 / 33
Técnico Sistemas Eléctricos (1°-3°)	41 / 42 / 43	41 / 42 / 43
Técnico Desarrollo de Software (1°-3°)	51 / 52 / 53	51 / 52 / 53

Además, configService.GRADOS_ARANCEL define los 23 nombres canónicos de grado y gradoCanónico() normaliza variantes de escritura ("5to Grado", "QUINTO GRADO" → "5° Grado") para buscar aranceles.

6. Mapa de rutas de la aplicación

6.1 Rutas públicas

Ruta	Página
/	Inicio (Home)
/nosotros	Nosotros (About)
/historia	Historia
/ubicacion	Ubicación
/autoridades	Autoridades
/mundo-salesiano	Mundo Salesiano

/modelo-educativo	Modelo Educativo
/directorio-institucional	Directorio
/noticias	Noticias
/calendario-academico	Calendario Académico
/politica-privacidad	Política de Privacidad
/proceso-inscripcion	Proceso de Inscripción
/solicitud-admision	Canje de código y solicitud de admisión
/mi-solicitud	Portal del aspirante (seguimiento del expediente)
/portal/examen/:evaluationId	Examen académico en línea
/reingreso/login	Login de reingreso (carnet + PIN)
/reingreso/formulario	Ratificación de datos del alumno
/reingreso/pasos	Pasos de matrícula (pago y contrato)
/reingreso/talonario	Talonario NPE imprimible
/imprimir-contrato	Contrato imprimible (compartido)

6.2 Rutas administrativas

Ruta	Protección
/admin/login	Pública (formulario de acceso)
/admin/dashboard	Cualquier admin autenticado
/admin/institucional	Rol editor_web o admin
/admin/recursos, /admin/solicitudes, /admin/colecturia, /admin/importar-alumnos, /admin/codigos, /admin/contratos, /admin/contratos-firmados, /admin/evaluaciones, /admin/evaluaciones/constructor, /admin/aprobacion	Rol admin + candado (re-autenticación, 30 min por pestaña)

7. Diccionario de datos (Firestore)

Tipos fuente: src/types/index.ts y src/types/reingreso.ts. Las fechas se guardan como Timestamp de Firestore y se convierten a Date en los servicios. Los campos con "?" son opcionales.

7.1 Resumen de colecciones

Colección	ID del documento	Escrito por	Propósito
admissions	Auto-ID	Aspirante + admin	Solicitud de admisión completa (expediente, dictamen, credenciales, pago, contrato).
accessCodes	Auto-ID	Admin (el aspirante lo consume)	Códigos de acceso al proceso de admisión.
admissionDocuments	Auto-ID	Aspirante + admin	Documentos del expediente con revisión y versiones.
admissionEvaluations	Auto-ID	Admin + aspirante (examen)	Evaluaciones individuales (cita, asistencia, examen, nota, resultado).
evaluationTemplates	Auto-ID	Admin	Plantillas del examen académico por grado (preguntas).
evaluationPhases / evaluators / evaluationSessions / globalEvaluationSchedules	Auto-ID	Admin	Configuración del proceso de evaluación (fases, evaluadores, jornadas, fechas globales por año).
enrollments / enrollmentPayments /	Auto-ID	Familia + admin	Proceso de matrícula formal. NOTA: flujo parcialmente sin uso

enrollmentContracts			(el pago/contrato real del aspirante vive HOY en el doc de admissions).
alumnos	Carnet (8 dígitos)	Admin (importación/alta) + el propio alumno	Expediente maestro del alumno (antiguo ingreso + aprobados de nuevo ingreso).
reingresoPayments	Carnet	Alumno + admin	Comprobante de pago y contrato firmado del reingreso.
studentCredentials	admissionId	Admin (vía approveForEnrollment)	Contraseña temporal M365 aislada (solo lectura de admins).
usuarios_admin	Correo en minúsculas	Manual (consola Firebase)	Rol de cada administrador: { rol: "admin" "editor_web" }.
configuracion	matricula (y docs de contrato)	Admin	Ciclo escolar activo, fecha límite, cuotas por defecto y aranceles por grado; plantilla del contrato.
institucional / noticias / eventos / autoridades	Varios	Admin/editor_web	Contenido del sitio público (lectura pública).

7.2 admissions (solicitud de admisión)

Campo	Tipo	Descripción
studentFirstName / studentLastName	string	Nombres y apellidos del aspirante.
dateOfBirth	Date	Fecha de nacimiento.
gender	'M' 'F'	Sexo.
previousSchool?	string	Centro de procedencia.
gradeApplying	string	Grado al que aplica.
parentFirstName/LastName/Email/Phone/Relationship	string	Datos del encargado que solicita.
linkedSiblingCarnet? / linkedSiblingName?	string	Hermano ya inscrito (si aplica).
departamento / distrito / municipio / direccion	string	Dirección (estructura de El Salvador, catálogo en src/data/elSalvadorGeo.ts).
howDidYouHear? / comments?	string	Mercadeo y comentarios.
status	'pending' 'evaluations' 'interview' 'approved' 'rejected' 'enrolled'	Estado macro de la solicitud.
applicationDate	Date	Fecha de solicitud.
expedienteDigital?	Record<string,string>	Expediente llenado por la familia (NIE, salud, sacramentos, familia, transporte, sostenedor). Mismos datos que un alumno antiguo.
expedienteCompletado?	boolean	Si ya completó el expediente.
privacyConsent?	{accepted, acceptedAt, policyVersion, acceptedByEmail?}	Consentimiento de datos personales (obligatorio, hay menores).
carnet?	string(8)	Carnet asignado al aprobar; se usa en el NPE.
paymentReceipt? / signedContract?	AdmissionFileReview	Comprobante de pago y contrato firmado subidos por la familia: {fileUrl, fileName, status pending/approved/rejected,

		uploadedAt, rejectionReason?, reviewedBy?, reviewedAt?}.
documents	string[]	Referencias de documentos (legado; el detalle vive en admissionDocuments).
accessCodeUsed? / enrollmentYear?	string / number	Código canjeado y año de matrícula (viene del código).
reviewedBy? / reviewedAt? / reviewNotes?	string/Date/string	Auditoría de revisión.
finalDecision?	objeto	Dictamen final: result, decidedBy(+Name), decidedAt, observations, rejectionReasons[], recommendations[], interviewDate, interviewNotes, canViewResults, rejectionMessage.
interviewResolution?	objeto	Resolución de la entrevista psicológica: decision, message, strengths[], areasToImprove[], recommendations, nextSteps, notifyParent, resolvedBy(+Name), resolvedAt.
assignedCredentials?	objeto	Credenciales: microsoftEmail, studentCode, assignedBy/At, firebaseUserId, microsoftUserId, teamsEnabled, welcomeEmailSent, provisioningStatus (not_started/pending/provisioned/failed), provisioningError, applyingForScholarship (beca Fundación LYRA), englishExam {scheduledDate/Time, location, completed, level...}. La contraseña ya NO se guarda aquí (ver studentCredentials).
institutionalPersonId? / psychologyCaselId?	string	Reservados para módulos futuros (hoy personRegistryService es un stub).

7.3 accessCodes (códigos de acceso)

Campo	Tipo	Descripción
code	string	Código que canjea la familia.
description?	string	Nota interna.

year	number	AÑO de matrícula que hereda la admisión.
gradeLevel	string	Grado para el que se emite.
maxUses / currentUses	number	Control de usos.
isActive	boolean	Si puede canjearse.
expiresAt?	Date	Expiración opcional.
createdAt / createdBy / creatorName?	Date / string	Auditoría de emisión.
assignedTo?	{applicantName, guardianName, guardianPhone, guardianEmail}	A quién se emitió (nominal).
usedBy?	{[name, grade, email, phone, usedAt]}	Registro de canjes.

7.4 admissionDocuments (documentos del expediente)

Campo	Tipo	Descripción
admissionId	string	Solicitud a la que pertenece.
type	birth_certificate previous_grades recent_photo vaccination_card identification financial_solveny grade_certificate other	Tipo de documento.
fileName / fileUrl / fileSize / mimeType	string / number	Archivo en Storage (admissions/{id}/...). Máx 10 MB, imagen o PDF.
status	pending approved rejected resubmitted	Estado de revisión.
reviewedBy? / reviewedAt? / rejectionReason?	string/Date/string	Auditoría de revisión.
version / previousVersions?	number / [{fileUrl, uploadedAt, rejectionReason?}]	Historial de re-subidas.
uploadedAt / updatedAt	Date	Marcas de tiempo.

7.5 admissionEvaluations (evaluaciones individuales)

Campo	Tipo	Descripción
admissionId / studentName	string	Aspirante evaluado.
phaseId / phaseType / phaseName	string / psychological academic english psychological_interview interview	Fase del proceso.
scheduledDate? / scheduledTime? / location?	Date / string	Cita programada.
evaluatorId? / evaluatorName?	string	Evaluador asignado.
attendanceStatus? (+MarkedAt/By/ByName)	pending present absent	Control de asistencia.
rescheduledCount? / autoScheduled? / manualAccessEnabled?...	number / boolean	Seguimiento operativo y acceso manual al examen.

status	pending scheduled in-progress completed cancelled	Estado.
score? / maxScore? / result?	number / approved reviewed needs-review rejected pending	Calificación.
observations? / recommendations? / notes?	string	Comentarios del evaluador.
examEnabled? / examStartedAt? / examCompletedAt?	boolean / Date	Examen en línea (fase académica).
subjectConfigs?	[[subjectId, subjectName, subjectIcon, enabled, questionCount]]	Materias activas del examen.
templateId? / answers?	string / Record<preguntaId, respuesta>	Plantilla usada y respuestas del alumno.
scheduledBy?/scheduledAt?/completedAt?/createdAt/updatedAt	Date/string	Metadatos.

Colecciones de apoyo: evaluationPhases {name, type, description, order, isRequired, passingScore?, maxScore?, estimatedDuration, isActive}; evaluators {name, email, type teacher | psychologist | admin, specialization?, canEvaluate[], isActive}; evaluationSessions (jornadas con capacidad); globalEvaluationSchedules {year, type, name, date, startTime, endTime, location, instructions?, evaluatorIds[], isActive} — fechas globales por año lectivo.

7.6 evaluationTemplates (plantillas de examen)

Campo	Tipo	Descripción
title / grade / subject?	string	Título, grado destino (ej. "Kinder 5") y resumen de materias.
type	'academic'	Único tipo actual.
isActive	boolean	Plantilla activa para su grado.
questions	[[{id, question, subject?, imageUrl?, options[], correctAnswer}]]	Preguntas de opción múltiple, con imagen de apoyo opcional.
createdBy / createdAt / updatedAt	string / Date	Auditoría.

7.7 alumnos (expediente maestro — AlumnoReingreso)

ID del documento = carnet (8 dígitos). Es la colección "de verdad" de estudiantes del colegio: contiene los 2,196 migrados de SQL más cada nuevo ingreso que completa su matrícula.

Campo	Tipo	Descripción
anioIngreso	string	Año en que ingresó al colegio.
gradoActual / gradoMatricular	string	Grado que cursa y grado al que se matricula (calculados; se avanzan solos al cambiar el ciclo).

anioCalculo?	number	Año para el que se calcularon los grados (base de importación: 2026).
pin	string	PIN de acceso al portal (= NIE; fallback últimos 4 del carnet). Solo se valida en el servidor.
estado	'ACTIVO' 'INACTIVO' 'RETIRADO'	Estado del alumno (INACTIVO no puede entrar).
carnet / nie / nombres / apellidos	string	Identidad.
sexo / fechaNac / nacionalidad	string	Datos personales (fechaNac 'YYYY-MM-DD').
zona / departamento / municipio / telefono / direccion	string	Dirección. OJO: departamento/municipio guardan CÓDIGOS crudos de SQL ("02", "0210") — falta catálogo (ver sección 9).
viveCon / religion / tipoSangre / enfermedades / alergias	string	Convivencia y salud.
bautizado / confirmado / comunión / cursoParvularia / centroProcedencia	string	Sacramentos y procedencia.
padre / madre / encargado	ContactoFamiliar	Cada uno: {nombre, lugarTrabajo, telefonoTrabajo, profesion, cargo, telefonoFijo, telefonoMovil, email, religion}.
responsable	string	Quién responde por el alumno.
emergencia	{llamarA, telefono}	Contacto de emergencia.
transporte	{tipo, nombreMotorista, placa, telefonoMotorista}	Transporte escolar.
facturacion	DatosFacturacion	{nombreCompleto, direccion, telefono, email, dui, nit, profesion, parentesco} del sostenedor económico. No venía de SQL: se precarga con el padre y la familia lo completa al ratificar.
correoInstitucional?	string	Correo M365 (solo alumnos que vinieron por admisión).
importadoDesdeSql? / createdAt? / updatedAt? / ratificadoAt?	boolean / Date	Metadatos; ratificadoAt = cuándo confirmó sus datos.

7.8 reingresoPayments (pagos y contratos de reingreso)

Campo	Tipo	Descripción
carnet (= ID del doc)	string	Alumno.
studentName / grade	string	Denormalizados para la bandeja de colecturía.
paymentReceipt	AdmissionFileReview	Comprobante de pago (Storage: reingreso/{carnet}/payment/...).
signedContract?	AdmissionFileReview	Contrato firmado (Storage: reingreso/{carnet}/contract/...).

7.9 configuracion/matricula (ciclo escolar y aranceles)

Campo	Tipo	Descripción
anioMatricula	number	Año lectivo que se está matriculando. TODO el sistema se ajusta a este valor (grados, talonarios, contratos, códigos).
fechaLimitePago	string 'YYYY-MM-DD'	Vencimiento del talonario NPE.
cuotaMatricula / cuotaMensualidad	number	Cuotas POR DEFECTO si el grado no tiene arancel propio (defaults de arranque: \$145 / \$85).
arancelesPorGrado?	Record<gradoCanonico, {cuotaMatricula, cuotaMensualidad}>	Cuotas específicas por grado; clave = nombre canónico de GRADOS_ARANCEL.

updatedAt? / updatedBy?	Date / string	Auditoría. Se edita en /admin/colecturia (pestañas "Ciclo Escolar" y "Precios y Cuotas por Grado").
-------------------------	---------------	---

En la misma colección "configuracion" viven además los datos de la plantilla del contrato (editados en /admin/contratos y leídos por ContratoImpresion). Otros documentos de contenido: institucional (misión/visión/contactos/portadas), noticias, eventos, autoridades — lectura pública, escritura solo admin.

7.10 Otras colecciones

Colección	Contenido
studentCredentials/{admissionId}	{ microsoftPassword } — contraseña temporal M365, aislada para que el aspirante no pueda leerla desde su admisión. Solo admins (reglas).
usuarios_admin/{correo}	{ rol: 'admin' 'editor_web' }. Se administra a mano en la consola de Firebase. Lectura solo admin, escritura bloqueada por reglas.
enrollments / enrollmentPayments / enrollmentContracts	Estructuras completas para matrícula formal (EnrollmentData con additionalInfo médica y contactos de emergencia, EnrollmentPayment por concepto, EnrollmentContract con contractNumber y datos del contrato). ATENCIÓN: createEnrollment y GRADE_FEES de enrollmentService no se invocan hoy — el flujo real usa admissions.paymentReceipt/signedContract y reingresoPayments.

7.11 Cloud Storage (estructura de archivos)

Ruta	Contenido	Reglas
admissions/{admissionId}/{categoria}/{archivo}	Documentos del expediente, comprobante de pago, contrato firmado (aspirantes).	Lectura pública; escritura sin cuenta, máx 10 MB, solo imagen o PDF.
reingreso/{carnet}/{payment contract}/{archivo}	Comprobantes y contratos firmados de reingreso.	Igual que arriba.
portadas/{archivo}	Imágenes de portada del sitio público (gestor web).	Lectura pública; escritura solo imágenes hasta 5 MB.
(todo lo demás)	—	Bloqueado por defecto.

8. Modelo de seguridad actual

8.1 Reglas de Firestore (firestore.rules)

- Default-deny: cualquier colección no contemplada está bloqueada.
- isAdmin() = usuario autenticado con proveedor 'password' (los alumnos entran con proveedor 'custom' y NO cuentan como admin).
- Contenido del sitio y configuración: lectura pública, escritura solo admin.
- alumnos: get/update solo del dueño (uid == carnet) o admin; LISTAR solo admin (evita volcar el directorio con PINs); crear/borrar solo admin.
- accessCodes: lectura y update públicos (el aspirante valida y consume su código); crear/borrar solo admin.

- admissions, admissionDocuments, evaluaciones, enrollments, reingresoPayments: lectura/creación/actualización públicas (los aspirantes operan sin sesión); BORRAR siempre solo admin.
- studentCredentials y usuarios_admin: solo admin.

8.2 Requisitos operativos para que el modelo funcione

12. En la consola de Firebase, Authentication → Settings: DESACTIVAR el registro público ("Enable create"). Si no, cualquiera podría crearse una cuenta y pasar por admin.
13. Crear el doc de usuarios_admin (o usar ADMIN_BOOTSTRAP en adminAuth.ts) para el primer admin, y cuentas editor_web para el personal de redes.
14. DESPLIEGUE COORDINADO: desplegar functions + hosting ANTES que firestore:rules (el cliente viejo se rompe con las reglas nuevas; el nuevo funciona con ambas). Orden: firebase deploy --only functions,hosting → probar login de reingreso → firebase deploy --only firestore:rules.

8.3 Debilidades conocidas y aceptadas (por diseño de la Fase actual)

- El flujo de ASPIRANTE opera sin autenticación: cualquiera que conozca/adivine el ID de una admisión puede leerla (contiene PII de menores). Mitigado por IDs no enumerables, pero la solución real es la "Fase 2c" (sección 9).
- Los archivos de Storage son de lectura pública si se conoce la ruta/URL.
- provisionM365User valida autenticación y dominio del correo (ADMIN_EMAIL_DOMAIN) pero no consulta el rol en usuarios_admin (defensa en profundidad pendiente; en la práctica solo un admin llega al botón).

9. Lo que FALTA (backlog para quien continúe)

9.1 Prioridad ALTA — Seguridad (Fase 2c)

Pendiente	Detalle	Dónde
Proteger el flujo del aspirante	Mover la validación/canje del código de acceso a una Cloud Function que emita una sesión (como ya se hizo con loginReingreso) y limitar las reglas de admissions/admissionDocuments/admissionEvaluations al dueño. Hoy son allow read/create/update: if true.	firestore.rules + functions/src/index.ts + accessCodeService
Verificar configuración de la consola	Confirmar que el registro público de Firebase Auth está desactivado y que las reglas versionadas (firestore.rules) están desplegadas en producción.	Consola Firebase
Rol dentro de provisionM365User	Además del dominio, verificar rol "admin" en usuarios_admin dentro de la función.	functions/src/index.ts
Storage con lectura restringida	Evaluar quitar allow read: if true de documentos/comprobantes (usar URLs firmadas o sesión).	storage.rules

9.2 Prioridad MEDIA — Datos y operación

Pendiente	Detalle
Catálogos geográficos	departamento/municipio/nacionalidad de los alumnos migrados guardan CÓDIGOS de SQL ('02', '0210'). Falta el catálogo que los traduzca a nombres (el de admisiones nuevas sí usa nombres, src/data/elSalvadorGeo.ts).

Alumnos "Graduado"	Decidir si se bloquea el login a los 998 graduados (hoy solo se bloquea estado INACTIVO).
Fotos de alumnos	La columna image de SQL Server no se migró.
Datos reales sensibles	Revisar la calidad de los 2,196 registros migrados (PIN = NIE: hay alumnos sin NIE que usan últimos 4 del carnet).
Limpieza de código muerto	enrollmentService.createEnrollment y GRADE_FEES no se usan; personRegistryService es un stub que devuelve IDs falsos; unificar o eliminar.
Imágenes de la página principal	Último commit del proyecto: "ya solo agregar mas imagenes a la paginas principal y estara al 100".

9.3 Prioridad MEDIA/LARGO PLAZO — Módulos no construidos

Los tipos ya existen en src/types/index.ts pero NO hay pantallas ni servicios reales para:

- Gestión académica: cursos (Course), calificaciones (Grade), asistencia (Attendance), docentes (Teacher con doble registro en systemUsers), estudiantes como entidad académica (Student con hasPortalAccess).
- Portal de estudiantes/padres: los tipos contemplan login compartido con contraseña y requirePasswordChange, pero no existe.
- Roles ampliados: UserRole define teacher, psychologist, secretary, accountant, hr, librarian, counselor, it_support, nurse, parent, student, scholarship — hoy solo se usan admin y editor_web.
- Cobro de MENSUALIDADES/colegiaturas: el sistema solo cubre el pago de matrícula. No hay estados de cuenta ni mora.
- Notificaciones por correo: existe el flag welcomeEmailSent pero nunca se envía nada (ni resultados, ni credenciales, ni recordatorios de pago).
- Gestión documental general (interface Document) y expediente psicológico (psychologyCaseld).
- Becas: applyingForScholarship (Fundación LYRA) solo se registra; no hay flujo de evaluación de becas.

9.4 Calidad de ingeniería

- No hay pruebas automatizadas (unitarias ni e2e).
- No hay CI/CD: el deploy es manual con firebase deploy.
- El README.md es el genérico de Vite — reemplazarlo con este documento o un extracto.
- No hay entorno de staging: se trabaja directo contra el proyecto de producción.
- Variables .env del frontend y functions/.env.* no están versionadas (correcto), pero deben respaldarse en un gestor de secretos del equipo.

10. Guía de traspaso (para el siguiente desarrollador)

10.1 Accesos que necesitas pedir

15. Proyecto de Firebase (gestion-escolar-cssj): rol Editor/Owner en la consola. Plan Blaze activo (necesario para Functions).

16. Azure AD del tenant salesianosan jose.edu.sv: App Registration usada por provisionM365User (client id/secret, permiso de aplicación User.ReadWrite.All con admin consent). El secret expira: calendarizar su rotación.
17. Repositorio Git (rama principal: main; trabajo actual en rama carlos).
18. Una cuenta admin en Firebase Auth + su doc en usuarios_admin con rol admin.
19. Archivo .env del frontend (VITE_FIREBASE_API_KEY, VITE_FIREBASE_AUTH_DOMAIN, VITE_FIREBASE_PROJECT_ID, VITE_FIREBASE_STORAGE_BUCKET, VITE_FIREBASE_MESSAGING_SENDER_ID, VITE_FIREBASE_APP_ID) — pedirlo al autor, no está en Git.

10.2 Puesta en marcha local

```
# Frontend
npm install
npm run dev          # Vite en http://localhost:5173

# Cloud Functions
cd functions
npm install
npm run build

# Lint / build de producción
npm run lint
npm run build        # tsc -b && vite build → dist/
```

10.3 Despliegue

```
npm run build
firebase deploy --only hosting          # sitio
firebase deploy --only functions       # backend
firebase deploy --only firestore:rules # reglas (SIEMPRE después de
functions+hosting)
firebase deploy --only storage         # reglas de Storage

# Secretos de Azure (solo la primera vez o al rotar)
firebase functions:secrets:set AZURE_TENANT_ID
firebase functions:secrets:set AZURE_CLIENT_ID
firebase functions:secrets:set AZURE_CLIENT_SECRET
```

10.4 Operación anual: abrir un nuevo ciclo escolar

20. Entrar a /admin/colecturia → pestaña "Ciclo Escolar": cambiar anioMatricula (ej. 2027 → 2028) y la fecha límite de pago.
21. Pestaña "Precios y Cuotas por Grado": actualizar aranceles si cambian.
22. Emitir los códigos de admisión del nuevo año en /admin/codigos.
23. NO hay que re-importar alumnos: sus grados se avanzan automáticamente al leer (anioCalculo vs ciclo activo).
24. Verificar la plantilla del contrato en /admin/contratos.

10.5 Mapa "quiero cambiar X → toco Y"

Quiero...	Archivo(s)
-----------	------------

Agregar/editar una ruta	src/App.tsx
Cambiar textos/estructura del sitio público	src/pages/home/* y src/components/home/* (el contenido dinámico se edita en /admin/institucional)
Cambiar el flujo de admisión	src/pages/admisiones/*, src/pages/admin/AdminAdmisiones.tsx, src/services/admissionService.ts
Cambiar la aprobación / credenciales / M365	src/pages/admin/AdminAprobacionMatricula.tsx, admissionService.approveForEnrollment, src/utls/credentialsGenerator.ts, functions/src/index.ts
Cambiar el examen en línea	src/pages/admin/evaluaciones/*, src/pages/admisiones/ExamenAspirante.tsx, src/services/evaluationService.ts
Cambiar reingreso / ratificación	src/pages/reingreso/*, src/services/alumnoService.ts, src/auth/studentSession.ts
Cambiar talonario / NPE / aranceles	src/pages/reingreso/TalonarioPrint.tsx, src/utls/npeGenerator.ts, src/services/configService.ts
Cambiar el contrato	src/pages/shared/ContratoImpresion.tsx, /admin/contratos (doc configuracion)
Cambiar roles/permisos del panel	src/auth/adminAuth.ts, src/components/layout/ProtectedRoute.tsx, firestore.rules
Importar/corregir alumnos migrados	src/pages/admin/AdminImportarAlumnos.tsx, src/utls/alumnoSqlParser.ts, src/data/alumnosParaImportar.ts

10.6 Convenciones y "gotchas" del proyecto

- Todo el dominio está en español (colecciones, comentarios, UI); los tipos de admisión heredados están en inglés. Mantener la mezcla con criterio: nuevo código de negocio en español.
- Hay DOS catálogos de códigos de grado (carnet vs NPE) — ver sección 5.1. No intercambiarlos.
- El año de matrícula NUNCA se quema en código: siempre configService.getConfigMatricula() o el hook useAnioMatricula.
- generateStudentCarnet debe seguir consultando TAMBIÉN la colección alumnos (hay carnets del SQL en el mismo rango).
- reingresoPayments y alumnos usan el CARNET como ID de documento; admissions usa auto-ID.
- La contraseña M365 es determinista (Cssj<carnet>!): las pantallas la recalculan, no la leen de la admisión.
- Cambios en firestore.rules exigen despliegue coordinado (sección 8.2, punto 3).
- El caché de configService dura toda la sesión del navegador: tras cambiar el ciclo escolar, recargar la página.

11. Anexo — Historial y estado a la fecha

- 2026-07-07: importados los 2,196 alumnos de SQL Server a Firestore (verificado en consola).
- 2026-07-08: ciclo escolar configurable, aranceles por grado, auditoría del flujo de nuevo ingreso, ciclo de vida completo aspirante→alumno.
- 2026-07-09: sistema de roles admin/editor_web + candado, reglas de Firestore versionadas, contraseñas M365 aisladas, login de reingreso server-side.
- Jul 2026: provisión M365 Fase 1 y 2 completas y probadas end-to-end (cuenta real creada con licencia A1 y Teams).

- Estado del repo al 2026-07-10: rama carlos, working tree limpio. Último commit: "ya solo agregar mas imagenes a la paginas principal y estara al 100".

Documento generado automáticamente a partir del código fuente el 10 de julio de 2026. Ante cualquier discrepancia, el código es la fuente de verdad.